❷ The type of event (domain-specific)

❸ The ID of the event

When you're done testing the application, stop the process with Ctrl-C.

NOTE You can refer to the Spring Cloud Function official documentation for more details on how HTTP, CloudEvents, and RSocket are supported (https://spring.io/projects/spring-cloud-function).

16.2.2 Deployment pipeline: Build and publish

Following the continuous delivery principles and techniques explained throughout the book, we can implement a deployment pipeline for Quote Service and Quote Function. Since the release candidate for those projects is a container image, most of the operations will be the same as for standard JVM applications.

When working locally, it's convenient to run and test serverless applications on the JVM rather than using GraalVM due to the shorter build time and the less resource-demanding process. However, to achieve better quality and catch errors earlier, we should run and verify the applications in native mode as early in the delivery process as possible. The commit stage is where we compile and test our applications, so it might be a good place to add those additional steps.

In your Quote Function project (quote-function), add a new .github/workflows folder, and create a commit-stage.yml file. As a starting point, you can copy the implementation of the commit stage from one of the other applications we built in the previous chapters, such as Catalog Service. The commit stage workflow we have used so far is composed of two jobs: "Build & Test" and "Package and Publish." We'll reuse the implementation from the other applications, but we'll add an intermediate job responsible for testing the native mode.